Rodrigo Fernandez

Senior AI Full Stack Engineer
Katy, Texas | +1 (407) 801-1287 | https://www.linkedin.com/in/rodrigo-bermejo-fern%C3%A1ndez-9a5a0664/ | rodrigobfdev@gmail.com

Summary

Senior AI Data Engineer with strong experience designing cloud-native data platforms for FinTech, healthcare, eCommerce, SaaS, and enterprise workflow industries using **Python 3.8–3.12**, **SQL, Apache Spark 3.x, Airflow 2.x**, dbt 1.x, Kafka 2.x–3.x, AWS, GCP, Snowflake, BigQuery, Docker, and modern LLM/RAG patterns, specializing in reliable ETL/ELT pipelines, feature-ready data models, vector search integration, observability, cost-aware orchestration, privacy-aware data governance, and production AI data workflows that convert raw operational data into trusted analytics, automation, and intelligent decision systems.

Skills

- **Programming & Data Engineering: Python 3.8–3.12**, SQL, PySpark, Bash, Pandas, NumPy, Pydantic, SQLAlchemy
- **AI / ML / LLM Data: LLM/RAG pipelines**, embeddings, vector search, feature engineering, model-ready datasets, OpenSearch Vector Search, pgvector, scikit-learn
- **Data Processing: Apache Spark 3.x**, Spark Structured Streaming, Kafka 2.x–3.x, Airflow 2.x, dbt 1.x, ETL/ELT, batch processing, streaming pipelines
- **Cloud & Warehousing: AWS S3**, Lambda, Glue, Redshift, CloudWatch, GCP BigQuery, Cloud Storage, Pub/Sub, Snowflake
- **Databases:** PostgreSQL, MySQL, SQL Server, Redis, DynamoDB, Elasticsearch/OpenSearch
- **DevOps & Quality:** Docker, Kubernetes, GitHub Actions, GitLab CI, Terraform, Great Expectations, OpenTelemetry, Prometheus, Grafana

Experience

Lightedge — Senior Software Engineer *(Apr 2020 – Mar 2026)*

- Architected **AI data pipelines** with **Python 3.8–3.12**, **Apache Spark 3.x**, **Airflow 2.x**, **dbt 1.x**, **Snowflake**, and **AWS S3**, supporting secure analytics for SaaS, infrastructure, and finance-facing reporting workloads.
- Built production **LLM/RAG data flows** by chunking operational documents, generating embeddings, storing vectors in **OpenSearch 2.x** and **pgvector 0.5–0.7**, and exposing trusted retrieval datasets for internal automation.
- Implemented a new **real-time anomaly detection** feature with **Kafka 2.8–3.6**, **Spark Structured Streaming 3.x**, and Python scoring jobs, reducing delayed incident discovery by **38%** across monitored customer data feeds.
- Resolved a recurring **Airflow 2.x scheduler failure** caused by oversized DAG parsing and unstable dynamic task generation by refactoring DAG factories, isolating configs, and cutting failed task retries by **46%**.
- Migrated legacy batch jobs from shell scripts and direct warehouse loads into modular **Python 3.x** packages, **dbt 1.x** models, and Airflow orchestration, improving release control and lowering manual recovery work.

- Designed privacy-aware **ETL/ELT** patterns with column masking, schema contracts, Great Expectations checks, and IAM-scoped storage access, helping sensitive customer datasets pass internal governance reviews.
- Optimized expensive **Snowflake** transformations by clustering high-volume tables, pruning incremental model windows, and tuning warehouse sizing, reducing monthly compute consumption by **29%** without weakening SLAs.
- Created reusable ingestion templates for APIs, SFTP drops, event streams, and warehouse extracts using **Python**, **Pydantic**, **Docker**, and CI checks, allowing teams to onboard new sources with fewer custom scripts.
- Fixed a production data quality bug where duplicate webhook events inflated revenue metrics by adding idempotency keys, late-arrival handling, and reconciliation dashboards across raw, staging, and curated layers.
- Delivered **feature engineering** datasets for churn, billing risk, and support-priority models using Spark aggregations, dbt marts, and ML-ready tables aligned with analyst definitions and model training requirements.
- Added observability with **OpenTelemetry**, CloudWatch, Prometheus metrics, Airflow alerts, and warehouse freshness checks, increasing pipeline failure visibility and helping teams respond before reports became stale.
- Collaborated with product, security, DevOps, and analytics teams to translate unclear business questions into governed data products, documented contracts, and production AI workflows that could evolve safely.

NIX United —Software Engineer *(Oct 2016 – Mar 2020)*

- Developed healthcare and eCommerce **data integration pipelines** with **Python 3.6–3.8**, **PostgreSQL 10–12**, **AWS S3**, **Redshift**, **Apache Spark 2.3–2.4**, and **Airflow 1.10** for multi-tenant client platforms.
- Implemented a patient appointment analytics pipeline that joined transactional records, timezone-normalized events, and notification logs, improving operational reporting accuracy by **34%** for scheduling teams.
- Fixed a critical **Spark 2.x memory error** caused by skewed joins on large order-history tables by salting high-cardinality keys, repartitioning datasets, and reducing job runtime by **41%**.
- Migrated cron-based CSV imports into **Airflow 1.10** DAGs with retries, task ownership, parameterized environments, and S3 landing zones, giving support teams clearer failure recovery paths.
- Built **Kafka 1.x–2.x** consumers for order, payment, and inventory events, then persisted validated payloads into PostgreSQL and Redshift models for near-real-time operational dashboards.
- Created backend-friendly data APIs with **Python Flask 1.x**, SQLAlchemy, and cached aggregate tables, helping React and PHP applications display reporting metrics without overloading transactional databases.
- Resolved duplicate customer-profile records by designing deterministic matching rules, checksum-based source tracking, and merge audit tables, improving downstream segmentation consistency by **27%**.
- Implemented incremental warehouse loading with watermarks, soft-delete detection, and SCD Type 2 dimensions, allowing analysts to compare historical account states without rebuilding full datasets.
- Improved data security by separating raw and curated buckets, tightening IAM roles, encrypting S3 objects, and removing hardcoded credentials from older deployment scripts.
- Supported model-ready datasets for fraud and product recommendation experiments using **scikit-learn 0.20–0.22**, pandas feature preparation, and SQL-based validation before handoff to data science teams.
- Partnered with QA and business analysts to validate edge cases in billing, appointments, and inventory reports, turning ambiguous defects into reproducible data fixes and regression checks.

Webiz Digital — Software Developer *(Jan 2014 – Sep 2016)*

- Built early **data engineering** workflows for digital commerce and marketing analytics using **Python 2.7–3.5**, **pandas 0.17–0.19**, **MySQL 5.6–5.7**, **PostgreSQL 9.x**, and Linux cron jobs.
- Created SQL reporting datasets that combined orders, campaign clicks, customer profiles, and payment statuses, helping business users measure conversion trends with fewer manual spreadsheet exports.
- Fixed an import bug where malformed CSV files broke nightly sales reports by adding schema validation, encoding normalization, and rejected-row logging for faster support investigation.
- Migrated repeated manual Excel cleanup into scripted **Python** transformations, reusable SQL views, and scheduled exports, reducing weekly analyst preparation effort by **31%**.
- Designed normalized MySQL tables and aggregate reporting views for product, customer, and order metrics, improving dashboard query speed while keeping application writes stable.
- Implemented basic predictive scoring prototypes with **scikit-learn 0.16–0.18**, logistic regression, and cleaned historical campaign data to support early lead-prioritization experiments.
- Resolved slow reporting queries by adding composite indexes, rewriting subqueries into joins, and archiving inactive records, improving high-traffic dashboard response time by **36%**.
- Added data reconciliation checks between payment exports and internal order tables, catching missing settlement rows before finance teams finalized client-facing reports.
- Documented repeatable ingestion steps, rollback notes, and SQL validation queries so developers could support data issues without waiting for the original script author.
- Worked flexibly across backend scripts, database tuning, lightweight APIs, and reporting support, building a practical foundation for later cloud-scale AI and analytics engineering.

Aurio — Junior Software Developer *(Oct 2012 – Dec 2013)*

- Supported internal business reporting systems with **Python 2.7**, **SQL Server 2008 R2–2012**, **MySQL 5.5**, stored procedures, and basic Linux automation for operations and customer-support teams.
- Cleaned customer, ticket, and transaction records using SQL scripts and small Python utilities, helping senior engineers prepare consistent datasets for monthly management reports.
- Fixed a recurring report mismatch caused by inconsistent timezone conversion by standardizing UTC storage rules and documenting the conversion logic used in downstream queries.
- Created stored procedures and indexed views for high-use operational reports, reducing repeated manual joins and making common customer-service metrics easier to retrieve.
- Assisted migration of legacy spreadsheet-based tracking into relational database tables with validation rules, import scripts, and review queries for business users.
- Built small automation scripts for log parsing, file movement, and scheduled report generation, reducing repetitive operational tasks and improving data availability each morning.
- Resolved duplicate ticket-count issues by tracing joins across customer and activity tables, then adding grouping logic and QA checks before report delivery.
- Learned production support discipline by checking failed jobs, reading logs, reproducing data defects, and escalating fixes with clear evidence for senior developers.
- Contributed to documentation for database tables, report fields, and known data issues, making junior support handoffs smoother and reducing repeated investigation time.

Microsoft — Software Developer Intern *(Jun 2012 – Sep 2012)*

- Supported internal software tooling with Python scripting, SQL queries, and backend debugging while learning enterprise engineering practices, source control, and structured code review.
- Assisted senior engineers with data validation scripts, API testing, and automation tasks that improved repeatable checks across internal development workflows.

- Gained practical experience in production-quality engineering, documentation, test discipline, and communication across distributed teams working on large-scale software systems.
- Resolved early-stage bugs related to inconsistent form validation and SQL query results by reproducing issues locally, checking logs, and working with mentors to apply safer input-handling rules.

Education

Texas State University
Bachelor's Degree in Computer Science (*Sep 2008 – May 2012)*